

8

The More You Know – Common Game Programming Patterns

As we move past the halfway point of this book, the bells and whistles in our games are going to get more and more advanced. To showcase them properly, the genre of our final project will be a classical 2D **Role Playing Game** with orthographic projection. With our code-base growing at a rapid rate, poor design quickly becomes tedious to maintain, or even unmanageable. As new features get added, we want expansion of code to be easy and not slow down the overall process. This is the area where game programming patterns shine the brightest.

In this chapter, we will be covering:

- The design and implementation of the entity component system
- Inter-system communication using the observer pattern
- Render ordering
- Implementation of map layers

Let's not waste any time and jump right into making our code base more robust!

Use of copyrighted resources

Once again, before beginning this chapter, we'd like to give credit where credit's due. The graphics used for the third project of this book consist of but are not limited to:

[LPC] Medieval fantasy character sprites by *wulax* under CC-BY-SA 3.0 and GPL 3.0 licenses:

<http://opengameart.org/content/lpc-medieval-fantasy-character-sprites>

Lots of free 2D tiles and sprites by *Hyptosis* under the CC-BY 3.0 license:

<http://opengameart.org/content/lots-of-free-2d-tiles-and-sprites-by-hyptosis>

All of the licenses that apply to the use of these resources can be found here:

- <http://creativecommons.org/licenses/by/3.0/>
- <http://creativecommons.org/licenses/by-sa/3.0/>
- <http://www.gnu.org/licenses/gpl-3.0.html>

What is a programming pattern?

Programming patterns, or design patterns, as they're more commonly referred to, are reusable and widely-implemented solutions to a given problem. That is not to say that these patterns exist as some sort of libraries out there, although there are libraries based on them. Instead, a programming pattern is more of an idea or a strategy. It is a well laid out plan on tackling a certain problem, the best possible answer to a given problematic situation, proven by time and experience, which is one of the the best reasons they should be used.

There are quite a few design patterns out there, as well as books, tutorials and even classes dedicated solely to understanding and implementing them. For our purposes, we're going to be covering four: the entity component system, event queue, observer and factory patterns. We'll be talking about each one separately, as they're non overlapping in function, even though they can be working together.

The entity component system

The **entity component system** is a programming pattern, which allows entities to possess properties and functionality through the means of composition, as opposed to inheritance. The biggest benefits of using this pattern include stronger decoupling of logic, easier serialization and de-serialization of entities, better reusability of code and ease of creating new entities. It does, however, add a fair bit of complexity to your code base.

The typical implementation of this pattern consists of three parts:

- **Entities:** In most cases, entities are barely anything more than identifiers, slapped on a collection of components
- **Components:** These are the building blocks of entities, that are nothing more than collections of data
- **Systems:** These are specialized classes that deal with a very specific task and are responsible for holding all of the logic in this paradigm

In addition to working with these three distinct types of elements, our entity component system is also going to need an entity manager to keep and manage all of the entity and component data, as well as the system manager, which will be responsible for updating each system, in addition to some other functionality we'll be covering soon.

In order to differentiate between different types of components and systems, we're going to create a new header file, `ECS_Types.h`, which will be used to store this information:

```
using ComponentType = unsigned int;
#define N_COMPONENT_TYPES 32

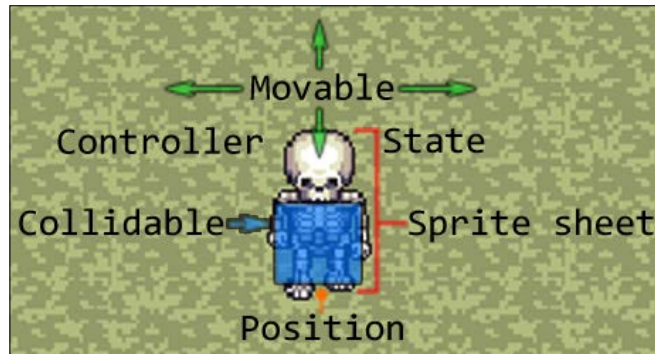
enum class Component{
    Position = 0, SpriteSheet, State, Movable, Controller, Collidable
};

enum class System{
    Renderer = 0, Movement, Collision, Control, State, SheetAnimation
};
```

In addition to component and system enumerations, we're also aliasing an unsigned integer to act as the component type and defining a macro `N_COMPONENT_TYPES`, which represents the maximum number of component types we can have.

What is a component?

Within the entity component system paradigm, a **component** is the smallest, non-overlapping aspect of an entity, such as its position, velocity or a sprite. From the programming point of view, however, it is nothing more than a simple data structure, which has no real logic in it. Its only job is storing information about the feature of an entity it represents, as illustrated below:



In order to store components easily, they have to rely on principles of inheritance. Let's take a look at a base component class definition:

```
class C_Base{
public:
    C_Base(const Component& l_type): m_type(l_type){}
    virtual ~C_Base(){}

    Component GetType(){ return m_type; }

    friend std::stringstream& operator >>(
        std::stringstream& l_stream, C_Base& b)
    {
        b.ReadIn(l_stream);
        return l_stream;
    }

    virtual void ReadIn(std::stringstream& l_stream) = 0;
protected:
    Component m_type;
};
```

The constructor of our component base class will take in the type of the component it represents. One thing to note is the overloaded >> operator, which calls a purely virtual function `ReadIn`. This serves as a quick way to read component data in from a file. Because each component is unique, it defines its own version of the `ReadIn` method in order to load its data correctly.

The position component

A good example of putting the base component class to work is actually implementing the first and arguably most common type of component: **position**.

```
class C_Position : public C_Base{
public:
    C_Position(): C_Base(Component::Position), m_elevation(0){}
    ~C_Position(){}

    void ReadIn(std::stringstream& l_stream){
        l_stream >> m_position.x >> m_position.y >> m_elevation;
    }

    const sf::Vector2f& GetPosition(){ return m_position; }
    const sf::Vector2f& GetOldPosition(){ return m_positionOld; }
    unsigned int GetElevation(){ return m_elevation; }

    void SetPosition(float l_x, float l_y){
        m_positionOld = m_position;
        m_position = sf::Vector2f(l_x,l_y);
    }

    void SetPosition(const sf::Vector2f& l_vec){
        m_positionOld = m_position;
        m_position = l_vec;
    }

    void SetElevation(unsigned int l_elevation){
        m_elevation = l_elevation;
    }

    void MoveBy(float l_x, float l_y){
        m_positionOld = m_position;
        m_position += sf::Vector2f(l_x,l_y);
    }
}
```

```
void MoveBy(const sf::Vector2f& l_vec) {
    m_positionOld = m_position;
    m_position += l_vec;
}

private:
    sf::Vector2f m_position;
    sf::Vector2f m_positionOld;
    unsigned int m_elevation;
};
```

The constructor of our component base class is invoked in the initializer list, with the component type being passed in as the only argument. Although there are better ways of assigning individual component types their own unique identifiers, it's better to start simple for clarity's sake.

This component keeps track of three pieces of data: its current position, the position it was at during the previous cycle, and the current elevation of an entity, which is simply a value that represents how high the entity is in relation to the map.

Much like any other component we will be covering later in the chapter, it offers a number of methods for modifying and obtaining its data members. While making its data members publically available is perfectly valid, offering helper methods reduces code redundancy and offers a familiar interface.

Lastly, note the implementation of the `ReadIn` method. It uses a `stringstream` object as an argument and loads the relevant pieces of data from it.

The bitmask

Having a lightweight, easy-to-use as well as easy-to-expend data structure, representing the makeup of any given entity, as well as a set of requirements imposed by a system saves a lot of headaches. For us, that data structure is a **bitmask**.



The standard template library provides its own version of a bitmask: the `std::bitset`. For educational purposes, we're going to be implementing our own version of this class.

As you probably know already, in binary, any and all numbers can be represented as a combination of zeroes and ones. However, who's to say that those two values have to be used only to represent a number? With some quick bitwise operator magic, any simple integer can be turned into a string of continuous flags that represent different aspects of an entity, such as which components it has, or types of components it needs to have in order to belong to a system.

Consider the following illustration:



The only real difference in practice would be a lot more than eight flags available. Let's get coding:

```
#include <stdint.h>

using Bitset = uint32_t;
class Bitmask{
public:
    Bitmask() : bits(0){}
    Bitmask(const Bitset& l_bits) : bits(l_bits){}

    Bitset GetMask() const{ return bits; }
    void SetMask(const Bitset& l_value){ bits = l_value; }

    bool Matches(const Bitmask& l_bits,
        const Bitset& l_relevant = 0)const
    {
        return(l_relevant ?
            ((l_bits.GetMask() & l_relevant) == (bits & l_relevant))
            : (l_bits.GetMask() == bits));
    }

    bool GetBit(const unsigned int& l_pos)const{
        return ((bits&(1 << l_pos)) != 0);
    }
    void TurnOnBit(const unsigned int& l_pos){
        bits |= 1 << l_pos;
    }
    void TurnOnBits(const Bitset& l_bits){
        bits |= l_bits;
    }
}
```

```
    }
    void ClearBit(const unsigned int& l_pos){
        bits &= ~(1 << l_pos);
    }
    void ToggleBit(const unsigned int& l_pos){
        bits ^= 1 << l_pos;
    }

    void Clear(){ bits = 0; }
private:
    Bitset bits;
};
```

We begin by defining the data type for our bitset, kindly provided by the `stdint.h` header. As the name implies, the `uint32_t` type is exactly 32 bits wide. Using this type, and not, let's say, a typical integer, eliminates the possibility of cross-platform differences. A regular integer could take up less or more memory, depending on the platform our code is executed on. Using specialized types from the `stdint.h` header ensures the exact same results, regardless of platform differences.

The majority of the `Bitmask` class consists of nothing but bitwise operations, which are an essential part of the C/C++ background. If you are not yet familiar with them, it's not the end of the world, however, it would be more beneficial to at least understand how they work before moving forward.

Managing entities

Now that we have the building blocks of entities defined, it's time to talk about storing and managing them. As mentioned previously, all an entity is at this point is a single identifier. Knowing that, we can begin shaping the way this data is going to be stored, beginning, as always, with the definition of data types to be used:

```
using EntityId = unsigned int;

using ComponentContainer = std::vector<C_Base*>;
using EntityData = std::pair<Bitmask, ComponentContainer>;
using EntityContainer = std::unordered_map<EntityId, EntityData>;
using ComponentFactory = std::unordered_map<
    Component, std::function<C_Base*(void)>>;
```

The first data type we'll be working with is the entity identifier, once again represented by an unsigned integer. Next, a container is needed to hold all of the components for an entity. A vector works just fine for this purpose. Following that, we define a pair, a bitmask and the component container, which will hold all of the information about the entity. The bitmask is used here in order to alleviate the need to iterate over containers searching for components, when it can be quickly queried for the same purpose. The last piece of the entity puzzle is mapping an entity identifier to all of its data, for which we'll be using the `unordered_map`.

In order to generate different component types with as little code as possible, we'll be using our trusty lambda-expression factory method here as well. The last four lines of type definitions here make that possible.

Having all of the data types defined allows us to finally take a look at the entity manager class declaration:

```
class SystemManager;
class EntityManager{
public:
    EntityManager(SystemManager* l_sysMgr,
        TextureManager* l_textureMgr);
    ~EntityManager();

    int AddEntity(const Bitmask& l_mask);
    int AddEntity(const std::string& l_entityFile);
    bool RemoveEntity(const EntityId& l_id);

    bool AddComponent(const EntityId& l_entity,
        const Component& l_component);

    template<class T>
    T* GetComponent(const EntityId& l_entity,
        const Component& l_component){ ... }

    bool RemoveComponent(const EntityId& l_entity,
        const Component& l_component);
    bool HasComponent(const EntityId& l_entity,
        const Component& l_component);

    void Purge();
private:
    template<class T>
    void AddComponentType(const Component& l_id){
        m_cFactory[l_id] = []() -> C_Base* { return new T(); };
    }
}
```

```
// Data members
unsigned int m_idCounter;
EntityContainer m_entities;
ComponentFactory m_cFactory;

SystemManager* m_systems;
TextureManager* m_textureManager;
};
```

In a fairly predictable fashion, we have all of the methods that would exist in any other class that serves as a container. Two different versions of adding entities are provided, one based on a bitmask passed in as an argument, and the other for loading an entity configuration from a file. The method for obtaining a component from a specific entity is templated, reducing the amount of code that has to be written outside of this class in order to obtain the type of component that is desired. Let's take a look at how it's implemented:

```
template<class T>
T* GetComponent(const EntityId& l_entity,
const Component& l_component)
{
    auto itr = m_entities.find(l_entity);
    if (itr == m_entities.end()) { return nullptr; }
    // Found the entity.
    if (!itr->second.first.GetBit((unsigned int)l_component))
    {
        return nullptr;
    }
    // Component exists.
    auto& container = itr->second.second;
    auto component = std::find_if(container.begin(), container.end(),
        [&l_component](C_Base* c) {
            return c->GetType() == l_component;
        });
    return (component != container.end() ?
        dynamic_cast<T*>(*component) : nullptr);
}
```

The entity argument passed into the method is evaluated first, in order to determine if one with the provided identifier exists. If it does, the bitmask of that entity is checked to verify that a component with the requested type is part of it. The component is then located in the vector and returned as the dynamically-cast type of the template.

Implementing the entity manager

With the class definition out of the way, we can start implementing its methods. As per usual, let's address the constructor and destructor of the entity manager class first:

```
EntityManager::EntityManager(SystemManager* l_sysMgr,
    TextureManager* l_textureMgr): m_idCounter(0),
    m_systems(l_sysMgr), m_textureManager(l_textureMgr)
{
    AddComponentType<C_Position>(Component::Position);
    AddComponentType<C_SpriteSheet>(Component::SpriteSheet);
    AddComponentType<C_State>(Component::State);
    AddComponentType<C_Movable>(Component::Movable);
    AddComponentType<C_Controller>(Component::Controller);
    AddComponentType<C_Collidable>(Component::Collidable);
}

EntityManager::~EntityManager() { Purge(); }
```

The constructor takes in a pointer to the `SystemManager` class, which we will be implementing shortly, as well as a pointer to the `TextureManager`. In its initializer list, the `idCounter` data member is set to zero. This is simply a variable that will be used to keep track of the last identifier that was given to an entity. Additionally, both the system manager and the texture manager pointers are stored for later reference. The last purpose of the constructor is adding all of the different types of components to the component factory.

The destructor simply invokes a `Purge` method, which will be used to clean up all of the dynamically allocated memory and clear all possible containers in this class.

```
int EntityManager::AddEntity(const Bitmask& l_mask) {
    unsigned int entity = m_idCounter;
    if (!m_entities.emplace(entity,
        EntityData(0, ComponentContainer())) .second)
    { return -1; }
    ++m_idCounter;
    for(unsigned int i = 0; i < N_COMPONENT_TYPES; ++i) {
        if(l_mask.GetBit(i)) { AddComponent(entity, (Component)i); }
    }
    // Notifying the system manager of a modified entity.
    m_systems->EntityModified(entity, l_mask);
    m_systems->AddEvent(entity, (EventID) EntityEvent::Spawned);
    return entity;
}
```

In the case of adding an entity based on the provided bitmask, a new entity pair is inserted into the entity container first. If the insertion was successful, a `for` loop iterates over all possible types of components and checks the mask for that type. The `AddComponent` method is then invoked, if the bitmask has the said type enabled.

After the component insertion, the system manager is notified of an entity being modified, or, in this case, inserted. The entity identifier, along with the bitmask of the said entity is passed into the `EntityModified` method of the system manager. An event is also created to alert the systems that this entity just spawned.

The identifier of the newly created entity is then returned. If the method failed to add an entity, `-1` is returned instead, to signify an error.

Removing an entity is every bit as easy, if not more so:

```
bool EntityManager::RemoveEntity(const EntityId& l_id){
    auto itr = m_entities.find(l_id);
    if (itr == m_entities.end()){ return false; }
    // Removing all components.
    while(itr->second.second.begin() != itr->second.second.end()){
        delete itr->second.second.back();
        itr->second.second.pop_back();
    }
    m_entities.erase(itr);
    m_systems->RemoveEntity(l_id);
    return true;
}
```

After the entity is successfully located in the entity container, the dynamically allocated memory of every single component it has is first freed, and the component is then removed from the vector. The entity itself is then erased from the entity container and the system manager is notified of its removal.

```
bool EntityManager::AddComponent(const EntityId& l_entity,
    const Component& l_component)
{
    auto itr = m_entities.find(l_entity);
    if (itr == m_entities.end()){ return false; }
    if (itr->second.first.GetBit((unsigned int)l_component))
    {
        return false;
    }
    // Component doesn't exist.
    auto itr2 = m_cFactory.find(l_component);
    if (itr2 == m_cFactory.end()){ return false; }
    // Component type does exist.
```

```

    C_Base* component = itr2->second();
    itr->second.second.emplace_back(component);
    itr->second.first.TurnOnBit((unsigned int)l_component);
    // Notifying the system manager of a modified entity.
    m_systems->EntityModified(l_entity, itr->second.first);
    return true;
}

```

Adding a component to an entity begins by verifying an entity with the provided identifier exists. If it does, and if there isn't already a component of that type added to the entity, the lambda-function container is queried for the desired type. Once the memory for the component is allocated, it's pushed into the component vector. The bitmask is then modified to reflect the changes made to the entity. The system manager is notified of those changes as well.

Predictably, a very similar process takes place when removing the component from an entity:

```

bool EntityManager::RemoveComponent(const EntityId& l_entity,
    const Component& l_component)
{
    auto itr = m_entities.find(l_entity);
    if (itr == m_entities.end()) { return false; }
    // Found the entity.
    if (!itr->second.first.GetBit((unsigned int)l_component))
    {
        return false;
    }
    // Component exists.
    auto& container = itr->second.second;
    auto component = std::find_if(container.begin(), container.end(),
        [&l_component](C_Base* c) {
            return c->GetType() == l_component;
        });
    if (component == container.end()) { return false; }
    delete (*component);
    container.erase(component);
    itr->second.first.ClearBit((unsigned int)l_component);

    m_systems->EntityModified(l_entity, itr->second.first);
    return true;
}

```

After confirming that both the entity and component exist, the memory allocated for the component is freed and the component itself is erased. The bitmask also gets modified to reflect these changes. Much like before, the system manager needs to know if an entity was altered, so the `EntityModified` method is invoked.

A fairly useful method to have for outside classes is one that checks if an entity has a certain type of component:

```
bool EntityManager::HasComponent(const EntityId& l_entity,
    const Component& l_component)
{
    auto itr = m_entities.find(l_entity);
    if (itr == m_entities.end()) { return false; }
    return itr->second.first.GetBit((unsigned int)l_component);
}
```

This follows the same pattern as before by checking if an entity exists first, then checking its bitmask for a certain component type.

It's cleanup time. Correctly disposing of all allocated resources is left up to the `Purge` method:

```
void EntityManager::Purge() {
    m_systems->PurgeEntities();
    for(auto& entity : m_entities) {
        for(auto &component : entity.second.second) { delete component; }
        entity.second.second.clear();
        entity.second.first.Clear();
    }
    m_entities.clear();
    m_idCounter = 0;
}
```

The system manager is notified to remove all of its entities first. While iterating over all of the entities in storage, it frees up the memory of every single component. The component container is then cleared. Lastly, the entity container itself is cleared and the identification counter is set back to 0.

The factory pattern

With complex data structures such as entities, chances are the programmer will not be setting up and initializing every single component by hand. Setting up entities with any arrangement of components quickly and with as little repeated code as possible is very much the goal here. Luckily, a programming pattern exists to solve this particular problem. It is simply referred to as the **factory** pattern.

The philosophy of use for this neat pattern is quite simple. There exists a class with some abstract method that takes in one or two arguments, pertaining to some vague identifying qualities. This class then generates, based on the information it was given, a class or a number of classes and returns a handle to them, effectively cutting out the part where data allocation or member initialization is done by hand. In other words, it is given a blueprint and produces a product based on it, hence the name "factory". This functionality was already achieved in a way by creating entities based on a bitmask, however, no actual data was initialized, only the defaults. Having a more pristine way of actually setting up these entities requires a more elaborate blueprint, so why not use text files? For example:

```
Name Player
Attributes 63
|Component|ID|Individual attributes|
Component 0 0 0 1
...
```

This format would allow everything there is to an entity to be stored in plain text as a blueprint and loaded at any time to produce any number of entities with exactly the same qualities. Let's take a look at how processing entity files could be achieved:

```
int EntityManager::AddEntity(const std::string& l_entityFile){
    int EntityId = -1;

    std::ifstream file;
    file.open(Utills::GetWorkingDirectory() +
        "media/Entities/" + l_entityFile + ".entity");
    if (!file.is_open()){
        std::cout << "! Failed to load entity: "
            << l_entityFile << std::endl;
        return -1;
    }
    std::string line;
    while(std::getline(file,line)){
        if (line[0] == '|'){ continue; }
        std::stringstream keystream(line);
        std::string type;
        keystream >> type;
        if(type == "Name"){

        } else if(type == "Attributes"){
            if (EntityId != -1){ continue; }
            Bitset set = 0;
            Bitmask mask;
            keystream >> set;
```

```
        mask.SetMask(set);
        EntityId = AddEntity(mask);
        if(EntityId == -1){ return -1; }
    } else if(type == "Component"){
        if (EntityId == -1){ continue; }
        unsigned int c_id = 0;
        keystoream >> c_id;
        C_Base* component = GetComponent<C_Base>
            (EntityId, (Component)c_id);
        if (!component){ continue; }
        keystoream >> *component;
        if(component->GetType() == Component::SpriteSheet){
            C_SpriteSheet* sheet = (C_SpriteSheet*)component;
            sheet->Create(m_textureManager);
        }
    }
}
file.close();
return EntityId;
}
```

Loading entity files isn't much different from any other files we've processed in the past. For now, reading entity names hasn't yet been implemented. The attributes line is simply the numeric value that the bitmask has with the desired components enabled. Once that value is read in, we pass it in to the other version of `AddEntity`, in order to create it and have all of the components properly allocated.

Reading in the actual components is slightly more complicated. First, we must make sure that the entity has been created. This means that the "Attributes" line has to come before the individual component data in the entity file. If the entity ID is greater than -1, we proceed with reading in the component ID and obtaining the actual object based on it. The overloaded `>>` operator comes in handy here, since it greatly simplifies actually streaming in the component data.

Lastly, due to the nature of resource handling, the component type has to be checked in order to provide its instance with a pointer to the texture manager class, if it needs it. We haven't yet created such components, however one of them will be the sprite sheet component that will represent some entities.

Designing the systems

With the data side of this paradigm being accounted for, the last remaining component remains to be the **system**. As the name loosely implies, systems are responsible for handling all of the logic that takes place inside and between components. Things ranging from sprites being rendered to collision checks are all handled by their own, respective systems to ensure complete separation between non-overlapping parts of the game. At least, that's how it should play out in an ideal world. In reality, as hard as one tries to decouple and categorize logic or data, some things still remain loosely connected, which is just the nature of the beast. Information still has to be traded between systems, however. Certain functionality also needs to be invoked as a consequence of a totally unrelated system's actions. To put it simply, there needs to be a way for systems to talk to each other without them knowing anything about how the other one works.

Entity events

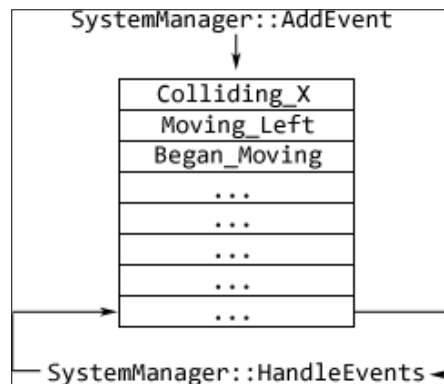
A fairly simplistic, and as it happens, necessary approach to handling inter-system relations is dispatching events. Every other system could be listening for those events and performing their own logic, completely separate from everything else, in response to a certain event taking place. Let's take a look at a possible list of entity events:

```
enum class EntityEvent{
    Spawned, Despawned, Colliding_X, Colliding_Y,
    Moving_Left, Moving_Right, Moving_Up, Moving_Down,
    Elevation_Change, Became_Idle, Began_Moving
};
```

This should give you a pretty good idea of how system communication will take place. Let's say that an entity is moving in the left direction. The "movement system" starts dispatching events, saying that it's in motion. The "animation system" listens for those events, and when they're received, it proceeds to increase frames in the entity's sprite sheet. Keep in mind that all of these chunks of logic are still completely separate from one another. The "movement system" is not increasing the frames of the entity's sprite sheet. It's simply saying to all of the other systems "Hello, I am moving entity x to the left," while they listen and react. It sounds like we can benefit from the "event queue."

Entity event queue

The **event queue** is a programming pattern, which is used to decouple when an event is fired from when it's actually processed. This illustration should capture the essence of it:



A queue is what's referred to as a First-In-First-Out data container. The data that is pushed onto it earliest is removed first. This serves our needs nicely. As the definition of the event queue states, its events are processed at a completely different time in relation to them being added. With that in mind, let's start designing the EventQueue class:

```
using EventID = unsigned int;

class EventQueue{
public:
    void AddEvent(const EventID& l_event){m_queue.push(l_event);}

    bool ProcessEvents(EventID& l_id){
        if (m_queue.empty()){ return false; }
        l_id = m_queue.front();
        m_queue.pop();
        return true;
    }

    void Clear(){ while(!m_queue.empty()){ m_queue.pop(); }}
private:
    std::queue<EventID> m_queue;
};
```

The event identifier is represented with an unsigned integer. In order to store the actual events, we'll be using, appropriately enough, a `queue` container. Adding an event to it is as simple as any other STL container. This class offers a method that can be used with while loops in order to simplify event processing. It returns a Boolean value in order to break the loop when the event queue is empty and its sole argument is taken by reference, in order to modify it each time the method is called. This is similar to the way SFML handles events.

The base system

In order to begin implementing our systems, they must first have a common base class, which not only provides a common interface that must be implemented, but also eliminates code redundancy. Much like most other classes we build, it will have its own data types defined:

```
using EntityList = std::vector<EntityId>;
using Requirements = std::vector<Bitmask>;
```

The system identifier, just like the component identifier, is represented by an unsigned integer. All of the entity identifiers will be stored in a vector container, much like the requirement bitmasks. The reason we would ever want to have more than one requirement bitmask is to have the ability to define combinations of different types of components that could still belong to the same system. A good example of that would be different drawable types belonging to the same rendering system.

Let's take a look at the header of our system base class:

```
class SystemManager;
class S_Base{
public:
    S_Base(const System& l_id, SystemManager* l_systemMgr);
    virtual ~S_Base();

    bool AddEntity(const EntityId& l_entity);
    bool HasEntity(const EntityId& l_entity);
    bool RemoveEntity(const EntityId& l_entity);

    System GetId();

    bool FitsRequirements(const Bitmask& l_bits);
    void Purge();

    virtual void Update(float l_dT) = 0;
    virtual void HandleEvent(const EntityId& l_entity,
        const EntityEvent& l_event) = 0;
```

```
protected:
    System m_id;
    Requirements m_requiredComponents;
    EntityList m_entities;

    SystemManager* m_systemManager;
};
```

We want every single system to have its own update method, as well as its own version of event handling. Additionally, systems having access to their own manager is also desired. Everything else that is not system-specific, like checking requirement bitmasks, is handled by the base class.

Implementing the base system

Because all systems require a pointer to the system manager, there's an issue of cross-inclusion. A forward declaration of it right before the class header and the inclusion of the system manager header in the implementation file takes care of the issue:

```
#include "System_Manager.h"
```

Time to go down the list of methods to implement, starting with the constructor and destructor:

```
S_Base::S_Base(const System& l_id, SystemManager* l_systemMgr)
    : m_id(l_id), m_systemManager(l_systemMgr) {}

S_Base::~S_Base() { Purge(); }
```

Each system has to have its own identifier, much like all of the components. That gets passed in the argument list to the constructor, along with a pointer to the system manager. Aside from setting the appropriate data members to these values in an initializer list, the constructor of a base system does nothing else.

The destructor, following the typical fashion, invokes the `Purge` method to do the cleanup.

```
bool S_Base::AddEntity(const EntityId& l_entity) {
    if (HasEntity(l_entity)) { return false; }
    m_entities.emplace_back(l_entity);
    return true;
}
```

Adding an entity to a system is fairly easy. If the identifier that was provided as an argument doesn't already exist within the said system, it simply gets pushed into the vector. How does a system determine if it has an entity with such an identifier? Let's find out:

```
bool S_Base::HasEntity(const EntityId& l_entity){
    return std::find(m_entities.begin(),
        m_entities.end(), l_entity) != m_entities.end();
}
```

Utilizing the `std::find` function allows us to sum up this method in a single line. Removing an entity also takes advantage of a similar function:

```
bool S_Base::RemoveEntity(const EntityId& l_entity){
    auto entity = std::find_if(m_entities.begin(), m_entities.end(),
        [&l_entity](EntityId& id){ return id == l_entity; });
    if (entity == m_entities.end()){ return false; }
    m_entities.erase(entity);
    return true;
}
```

In this case, we're using the `std::find_if` function, which takes a predicate as the third argument. A predicate is just another function that compares two elements together in order to find a match. In this case, we simply construct a lambda function that takes in an `EntityId` and returns a Boolean value, which will tell the find function whether a match was found. If it was, the entity is then removed.

Each system has to make sure that an entity has all the required components in order to be added to it. That's where this method comes in:

```
bool S_Base::FitsRequirements(const Bitmask& l_bits){
    return std::find_if(m_requiredComponents.begin(),
        m_requiredComponents.end(), [&l_bits](Bitmask& b){
        return b.Matches(l_bits, b.GetMask());
    }) != m_requiredComponents.end();
}
```

It takes in a bitmask as an argument and utilizes the same `std::find_if` function in conjunction with a lambda to locate a match. Very few systems will even need to define more than one bitmask for its required components, however it's nice to have this functionality when the need for it arises.

Lastly, here's the method for cleaning up:

```
void S_Base::Purge(){ m_entities.clear(); }
```

Because there's no actual dynamic memory being allocated here, it's safe to just empty the container of all the entity identifiers.

Handling messages

Entity events, while useful for a lot of situations, aren't perfect for everything. For instance, carrying data between systems is impossible using the event queue. The events are also being delivered to every single system, which can be wasteful. Instead, why not have an additional method of communication that not only carries data around, but also allows systems to pick and choose what they want to receive? Entity component system messaging serves exactly that purpose, and there just so happens to be yet another programming pattern, which allows easy implementation of the message-subscription approach.

The observer pattern

As the name entails, the **observer** pattern allows its users to pick and choose what they will be notified of. In other words, the observer will lay dormant after subscribing to information types it wishes to receive, and will only be notified if those types are encountered. Let's take a look at a very basic implementation of the Observer base class:

```
class Observer{
public:
    virtual ~Observer(){}
    virtual void Notify(const Message& l_message) = 0;
};
```

The Observer class is simply an interface, the inheritors of which must define a single method in order to use it. Although it looks simple, many features we desire in our game wouldn't be possible without it. Let's take a look at what these observers will be notified with:

```
using MessageType = unsigned int;
struct TwoFloats{ float m_x; float m_y; };

struct Message{
    Message(const MessageType& l_type) : m_type(l_type){}

    MessageType m_type;
    int m_sender;
    int m_receiver;

    union{
        TwoFloats m_2f;
        bool m_bool;
        int m_int;
    };
};
```

Aside from holding the information about the sender and receiver entities, as well as the message type, it employs a union in order to avoid inheritance. That essentially means that all of the data members within this union will be sharing the same space in memory, and only one of them can be valid at a time.

The last piece of the puzzle is containing all possible observers in a `Communicator` class. For this, we'll be using a vector:

```
using ObserverContainer = std::vector<Observer*>;
```

Because this class has relatively simple methods that simply deal with managing a vector container, let's take a look at the full class definition head to toe:

```
class Communicator{
public:
    ~Communicator(){ m_observers.clear(); }

    bool AddObserver(Observer* l_observer){
        if (HasObserver(l_observer)){ return false; }
        m_observers.emplace_back(l_observer);
        return true;
    }

    bool RemoveObserver(Observer* l_observer){
        auto observer = std::find_if(m_observers.begin(),
            m_observers.end(), [&l_observer](Observer* o){
                return o == l_observer; });
        if (observer == m_observers.end()){ return false; }
        m_observers.erase(observer);
        return true;
    }

    bool HasObserver(const Observer* l_observer){
        return (std::find_if(m_observers.begin(), m_observers.end(),
            [&l_observer](Observer* o){
                return o == l_observer;
            }) != m_observers.end());
    }

    void Broadcast(const Message& l_msg){
        for(auto& itr : m_observers){ itr->Notify(l_msg); }
    }

private:
    ObserverContainer m_observers;
};
```

The basic methods for adding, removing and looking for an observer are all typical. One thing to note, however, is the `Broadcast` method, which simply invokes the `Notify` method of an observer and passes in a message to send.

Last, and definitely the least amount of code is responsible for actually putting the observer approach to use:

```
class S_Base : public Observer{ ... }
```

Because the base system class has virtual methods, it doesn't need to implement its own version of `Notify`. That will be the job for all the systems that inherit from this class.

The message handler class

We have all of the pieces to build a cohesive messaging system. Let's take a look at some data types that will be used to store message subscription information:

```
using Subscriptions = std::unordered_map<
    EntityMessage, Communicator>;
```

Each possible message type will have its own communicator that will broadcast the message to all of its observers. Using an `unordered_map` is perfect for expressing such a relationship.

The message handler is a very simple class, so let's take a look at its entire implementation:

```
class MessageHandler{
public:
    bool Subscribe(const EntityMessage& l_type,
        Observer* l_observer)
    {
        return m_communicators[l_type].AddObserver(l_observer);
    }

    bool Unsubscribe(const EntityMessage& l_type,
        Observer* l_observer)
    {
        return m_communicators[l_type].RemoveObserver(l_observer);
    }

    void Dispatch(const Message& l_msg){
        auto itr = m_communicators.find(
            (EntityMessage)l_msg.m_type);
```

```

        if (itr == m_communicators.end()) { return; }
        itr->second.Broadcast(l_msg);
    }
private:
    Subscriptions m_communicators;
};

```

Subscribing and unsubscribing to message types is simply done by manipulating the unordered map data container. When a message is dispatched, the message type in the subscription container is queried. If it is found, the communicator's Broadcast method is invoked with the message passed in as the argument.

At this point you might be wondering what kind of messages we'll be handling. Let's take a gander at the EntityMessages.h file:

```

enum class EntityMessage{
    Move, Is_Moving, State_Changed, Direction_Changed,
    Switch_State, Attack_Action, Dead
};

```

The purpose of a messaging system quickly becomes clear, even by simply reading the names of message types. Every one of them lends itself to either needing to contain extra data or only ever applying to a single system.

Managing systems

Finally, we've arrived at the last stop on the entity component system route: handling systems themselves. Let's quickly review our custom data types for this class:

```

using SystemContainer = std::unordered_map<System, S_Base*>;
using EntityEventContainer = std::unordered_map<
    EntityId, EventQueue>;

```

The first data type, SystemContainer, is really hard to misinterpret. An unordered map is used to link system identifiers to actual systems. The second type definition here is responsible for storage of entity events. It also uses an unordered map and links entity identifiers to EventQueue instances, that all hold events for a specific entity until they're processed.

It's time to design the system manager class:

```

class EntityManager;
class SystemManager{
public:
    SystemManager();

```

```
~SystemManager();

void SetEntityManager(EntityManager* l_entityMgr);
EntityManager* GetEntityManager();
MessageHandler* GetMessageHandler();

template<class T>
T* GetSystem(const System& l_system){...}

void AddEvent(const EntityId& l_entity, const EventID& l_event);

void Update(float l_dT);
void HandleEvents();
void Draw(Window* l_wind, unsigned int l_elevation);

void EntityModified(const EntityId& l_entity,
    const Bitmask& l_bits);
void RemoveEntity(const EntityId& l_entity);

void PurgeEntities();
void PurgeSystems();
private:
    SystemContainer m_systems;
    EntityManager* m_entityManager;
    EntityEventContainer m_events;
    MessageHandler m_messages;
};
```

As expected, it needs to have methods for adding and handling events, updating and drawing the systems, notifying them of entity changes and removal requests, and obtaining them as well. The template method for getting a particular system is implemented this way:

```
template<class T>
T* GetSystem(const System& l_system){
    auto itr = m_systems.find(l_system);
    return(itr != m_systems.end() ?
        dynamic_cast<T*>(itr->second) : nullptr);
}
```

Just like the entity manager's method of obtaining components, this method relies on the use of templates and dynamic casting in order to obtain a system in the correct form.

Implementing the system manager

Cross-inclusion peaks its ugly head once more, so we must combat it with forward declarations and inclusions of headers in the implementation files:

```
#include "Entity_Manager.h"
```

With that out of the way, we can now start implementing the constructor and destructor:

```
SystemManager::SystemManager(): m_entityManager(nullptr) {
    m_systems[System::State] = new S_State(this);
    m_systems[System::Control] = new S_Control(this);
    m_systems[System::Movement] = new S_Movement(this);
    m_systems[System::Collision] = new S_Collision(this);
    m_systems[System::SheetAnimation] = new S_SheetAnimation(this);
    m_systems[System::Renderer] = new S_Renderer(this);
}

SystemManager::~SystemManager() {
    PurgeSystems();
}
```

The constructor sets up a pointer to an entity manager class before initializing all of the systems it holds. The destructor performs its usual job of cleaning up the mess, which is entrusted to the `PurgeSystems` method.

Because the system manager needs to have a pointer to the entity manager and vice versa, the one that's instantiated first will not simply be able to take a pointer to the other class in its constructor, hence the need for the `SetEntityManager` method:

```
void SystemManager::SetEntityManager(EntityManager* l_entityMgr) {
    if(!m_entityManager) { m_entityManager = l_entityMgr; }
}
```

With a class so broadly applied, it needs to provide getter methods for its data members:

```
EntityManager* SystemManager::GetEntityManager() {
    return m_entityManager;
}

MessageHandler* SystemManager::GetMessageHandler() {
    return &m_messages;
}
```

This ensures that all systems have access to the message handler, as well as the entity handler.

Speaking of system access, they also must be able to add events to any entity:

```
void SystemManager::AddEvent(const EntityId& l_entity,
    const EventID& l_event)
{
    m_events[l_entity].AddEvent(l_event);
}
```

Using an `unordered_map` structure here really makes this method simple and neat. The entity identifier being the key, it's easy to access its individual event queue and add to it.

Providing we want those systems to tick, an update loop is in order:

```
void SystemManager::Update(float l_dT) {
    for(auto &itr : m_systems) {
        itr.second->Update(l_dT);
    }
    HandleEvents();
}
```

Here, every single system's update method is invoked and the elapsed time is passed in. The event handling takes place after all of the systems have been updated. Time to dissect that method:

```
void SystemManager::HandleEvents() {
    for(auto &event : m_events) {
        EventID id = 0;
        while(event.second.ProcessEvents(id)) {
            for(auto &system : m_systems)
            {
                if(system.second->HasEntity(event.first)) {
                    system.second->HandleEvent(event.first, (EntityEvent)id);
                }
            }
        }
    }
}
```

We begin by iterating over the event queues of different entities. An event identifier variable is set up and used in a `while` loop by reference, in order to obtain information from the queue. Every system in the manager is iterated over and checked for having the entity of interest. If it does, the system's `HandleEvent` method is invoked and the relevant information is passed in. That, in a nutshell, concludes event management on a larger scale. Now every individual system only has to worry about which events it wants to handle and how it wants to respond to them.

In order to populate the dark void of the screen with entities, we're going to need a `Draw` method:

```
void SystemManager::Draw(Window* l_wind,
    unsigned int l_elevation)
{
    auto itr = m_systems.find(System::Renderer);
    if (itr == m_systems.end()) { return; }
    S_Renderer* system = (S_Renderer*)itr->second;
    system->Render(l_wind, l_elevation);
}
```

For most needs, having a single system devoted to rendering entities more than suffices. Ergo, the renderer system is located in the system container and type-cast up from the base class. Its `Render` method is then invoked with relevant arguments, one of which is the current elevation that's being rendered. Drawing it this way allows the feel of "depth" to be achieved within our game.

As entities are not static in their composure, systems must be aware of these changes and properly take in or dispose of them, given the circumstances. This specific method has already been brought up multiple times during the implementation of the entity manager class, so let's take a look at how it works:

```
void SystemManager::EntityModified(const EntityId& l_entity,
    const Bitmask& l_bits)
{
    for(auto &s_itr : m_systems) {
        S_Base* system = s_itr.second;
        if(system->FitsRequirements(l_bits)) {
            if(!system->HasEntity(l_entity)) {
                system->AddEntity(l_entity);
            }
        } else {
            if(system->HasEntity(l_entity)) {
                system->RemoveEntity(l_entity);
            }
        }
    }
}
```

Upon any changes regarding entities taking place, the `EntityModified` method must be invoked with the identifier of the entity and its new bitmask passed in as arguments. Every system is then iterated over. Their respective `FitsRequirements` methods are invoked with the new bitmask as the argument. If the entity fits the requirements of a system and it doesn't belong to it, it is added. If, however, the entity does not fit these requirements but a system still has this entity, it's removed. The use of this simple concept allows entities to be dynamic in structure. Any given entity can lose or gain a component and immediately "transform" into something else.

Removal of entities is quite simple:

```
void SystemManager::RemoveEntity(const EntityId& l_entity) {
    for(auto &system : m_systems) {
        system.second->RemoveEntity(l_entity);
    }
}
```

All that needs to happen here is the `RemoveEntity` method of every system being invoked, which is quite similar to purging all entities:

```
void SystemManager::PurgeEntities() {
    for(auto &system : m_systems) {
        system.second->Purge();
    }
}
```

Getting rid of all systems in the system manager is also a cake walk:

```
void SystemManager::PurgeSystems() {
    for (auto &system : m_systems) {
        delete system.second;
    }
    m_systems.clear();
}
```

Because systems are dynamically allocated, the memory has to be freed for each of them. The system container is then simply cleared.

This last method marks the completion of our system manager, as well as the core structure of the entity component system paradigm. All of the basic tools for shaping our game are now present, so let's implement the first and most important system in the game: the renderer.

Implementing the rendering system

In order for entities to be drawn on screen, they must have a component that represents them visually. After some careful planning, one can deduce that an entity will probably not have just one possible choice for a graphical representation. For example, instead of a sprite sheet, an entity can be a simple shape with a single color fill. In order to make that happen, we need a common interface for drawable components. Let's see what we can come up with:

```
class C_Drawable : public C_Base{
public:
    C_Drawable(const Component& l_type) : C_Base(l_type){}
    virtual ~C_Drawable(){}

    virtual void UpdatePosition(const sf::Vector2f& l_vec) = 0;
    virtual const sf::Vector2u& GetSize() = 0;
    virtual void Draw(sf::RenderWindow* l_wind) = 0;
private:

};
```

The first thing to note here is that the constructor of this class also takes in a component type, and simply passes it to the base class. Since `C_Drawable` only has purely virtual methods, it can never be instantiated and will only be used as a mold to shape other drawable components. It requires all derived classes to implement a method for updating the drawable's position, obtaining its size and drawing it on screen.

The sprite sheet component

With the base class set up, it's time to take a look at creating the sprite sheet component:

```
class C_SpriteSheet : public C_Drawable{
public:
    ...
private:
    SpriteSheet* m_spriteSheet;
    std::string m_sheetName;
};
```

This component will, of course, be utilizing the `SpriteSheet` class we built in the past as one of its data members. We also want to hold on to the sprite sheet name in order to properly allocate resources after the time of de-serialization. Let's begin implementing the sprite sheet component:

```
C_SpriteSheet() : C_Drawable(Component::SpriteSheet),
    m_spriteSheet(nullptr) {}

~C_SpriteSheet() {
    if(m_spriteSheet) { delete m_spriteSheet; }
}
```

So far, nothing is out of the ordinary. The constructor uses the initializer list to set up the component type and set the sprite sheet pointer to `NULL`, while the destructor takes care of de-allocating the memory that's taken up by said sprite sheet.

Next, let's handle reading in the component data, which only consists of the sprite sheet name:

```
void ReadIn(std::stringstream& l_stream) {
    l_stream >> m_sheetName;
}
```

Due to the nature of this particular drawable component, it needs access to the texture manager. In order to set up the sprite sheet properly, the `Create` method is introduced:

```
void Create(TextureManager* l_textureMgr,
    const std::string& l_name = "")
{
    if (m_spriteSheet) { return; }
    m_spriteSheet = new SpriteSheet(l_textureMgr);
    m_spriteSheet->LoadSheet("media/Spritesheets/" +
        (l_name != "" ? l_name : m_sheetName) + ".sheet");
}
```

As seen previously, this particular method is used to set up the sprite sheet component during entity loading. It first checks if the memory for the `m_spriteSheet` data member hasn't already been allocated. If it hasn't, a new `SpriteSheet` object is created with the texture manager pointer passed in as its sole argument. The rest of the code deals with the second, optional argument. The name of the texture can be passed to the `Create` method itself, or it can use the `m_sheetName` data member that has been read in from the entity file.

Lastly, all of the virtual methods of the `C_Drawable` class must be implemented here:

```
SpriteSheet* GetSpriteSheet(){ return m_spriteSheet; }
void UpdatePosition(const sf::Vector2f& l_vec){
    m_spriteSheet->SetSpritePosition(l_vec);
}

const sf::Vector2u& GetSize(){
    return m_spriteSheet->GetSpriteSize();
}
void Draw(sf::RenderWindow* l_wind){
    if (!m_spriteSheet){ return; }
    m_spriteSheet->Draw(l_wind);
}
```

All of the work done on the `SpriteSheet` class in the past makes this pretty easy. One thing to note is that due to the nature of loading sprite sheet components, it may be wise to check if it has actually been allocated, before attempting to draw it.

The renderer

With the simple part out of the way, let's focus on creating our first system ever built, the renderer:

```
class S_Renderer : public S_Base{
public:
    S_Renderer(SystemManager* l_systemMgr);
    ~S_Renderer();

    void Update(float l_dT);
    void HandleEvent(const EntityId& l_entity,
        const EntityEvent& l_event);
    void Notify(const Message& l_message);
    void Render(Window* l_wind, unsigned int l_layer);
private:
    void SetSheetDirection(const EntityId& l_entity,
        const Direction& l_dir);
    void SortDrawables();
};
```

The header for all of the other systems will look very much like this one, with the exception of private methods, specific to the function that each individual system performs. Each system must implement its own `Update` and `HandleEvent` methods. Additionally, being an observer requires a unique implementation of the `Notify` method as well. Time to take a stab at implementing the renderer system:

```
S_Renderer::S_Renderer(SystemManager* l_systemMgr)
: S_Base(System::Renderer, l_systemMgr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::Position);
    req.TurnOnBit((unsigned int)Component::SpriteSheet);
    m_requiredComponents.push_back(req);
    req.Clear();

    m_systemManager->GetMessageHandler()->
        Subscribe(EntityMessage::Direction_Changed, this);
}
S_Renderer::~S_Renderer() {}
```

After invoking the base system's constructor and passing in the appropriate type, along with a pointer to the system manager, the renderer sets up a bitmask of requirements an entity has to meet in order to belong to this system. As you can see, it only needs to have the position and sprite sheet components. Once the requirement bitmask is added to the system, it also subscribes to the `Direction_Changed` message type. This utilizes the observer pattern discussed previously.

Let's take a look at the update method:

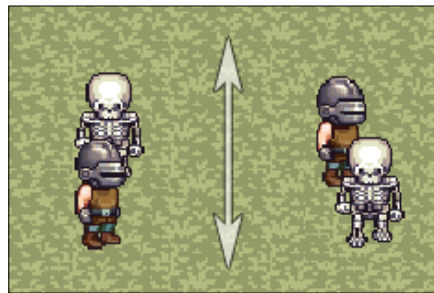
```
void S_Renderer::Update(float l_dT) {
    EntityManager* entities = m_systemManager->GetEntityManager();
    for(auto &entity : m_entities)
    {
        C_Position* position = entities->
            GetComponent<C_Position>(entity, Component::Position);
        C_Drawable* drawable = nullptr;
        if (entities->HasComponent(entity, Component::SpriteSheet)) {
            drawable = entities->
                GetComponent<C_Drawable>(entity, Component::SpriteSheet);
        } else { continue; }
        drawable->UpdatePosition(position->GetPosition());
    }
}
```

During iteration over all of the entities that belong to this system, the position and drawable components are obtained through the entity manager. The drawable's position is then updated through the use of its `UpdatePosition` method. This method can obviously be expanded if additional drawable types are added in the future.

Next, let's handle the appropriate events:

```
void S_Renderer::HandleEvent(const EntityId& l_entity,
    const EntityEvent& l_event)
{
    if (l_event == EntityEvent::Moving_Left ||
        l_event == EntityEvent::Moving_Right ||
        l_event == EntityEvent::Moving_Up ||
        l_event == EntityEvent::Moving_Down ||
        l_event == EntityEvent::Elevation_Change ||
        l_event == EntityEvent::Spawned)
    {
        SortDrawables();
    }
}
```

If the system encounters any events of entities spawning, changing their position or elevation, their drawable representations have to be re-sorted in order to assure the correct layering. The result of this is quite worth the trouble:



The code for message handling here is as follows:

```
void S_Renderer::Notify(const Message& l_message) {
    if (HasEntity(l_message.m_receiver)) {
        EntityMessage m = (EntityMessage)l_message.m_type;
        switch (m) {
            case EntityMessage::Direction_Changed:
                SetSheetDirection(l_message.m_receiver,
                    (Direction)l_message.m_int);
        }
    }
}
```

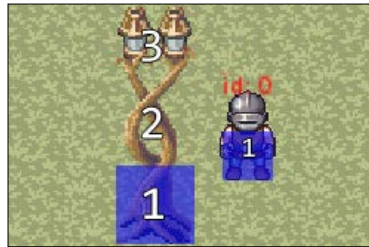
```
        break;
    }
}
}
```

Since messages are globally broadcasted to every system regardless of what entities they have, and the renderer only deals with a single message type that pertains to specific entities only, a check takes place in order to make sure the entity exists within the renderer system. The sole message type we care about so far is the direction being altered, in which case a private method is invoked to adjust it.

Now, let's address the main purpose the renderer system exists:

```
void S_Renderer::Render(Window* l_wind, unsigned int l_layer)
{
    EntityManager* entities = m_systemManager->GetEntityManager();
    for(auto &entity : m_entities){
        C_Position* position = entities->
            GetComponent<C_Position>(entity, Component::Position);
        if(position->GetElevation() < l_layer){ continue; }
        if(position->GetElevation() > l_layer){ break; }
        C_Drawable* drawable = nullptr;
        if (!entities->HasComponent(entity,
            Component::SpriteSheet))
        {
            continue;
        }
        drawable = entities->
            GetComponent<C_Drawable>(entity, Component::SpriteSheet);
        sf::FloatRect drawableBounds;
        drawableBounds.left = position->GetPosition().x -
            (drawable->GetSize().x / 2);
        drawableBounds.top = position->GetPosition().y -
            drawable->GetSize().y;
        drawableBounds.width = drawable->GetSize().x;
        drawableBounds.height = drawable->GetSize().y;
        if (!l_wind->GetViewSpace().intersects(
            drawableBounds))
        {
            continue;
        }
        drawable->Draw(l_wind->GetRenderWindow());
    }
}
```

Each entity is iterated over, much like any other system. The main difference here is the layer argument the method takes in. Because we want to have a map that has many different layers entities can be, for the lack of better term, "sandwiched" in, rendering elevation by elevation is necessary in order to maintain the correct draw order and offer the feeling of depth, as illustrated here:



The second half of the `Render` method is devoted to entity culling. First, a rectangle structure is created in order to represent the boundaries of a drawable. In the case of a sprite sheet, we know that its origin is set at the point of half its width and its full height. Using this information, the rectangle structure is properly set up and inspected for intersections with the view space, which essentially means the sprite is on screen and should be drawn.

Re-using code as much as possible makes life easier in the long run, hence the existence of many private methods amongst multiple systems, regarding modifications of entity components. For example:

```
void S_Renderer::SetSheetDirection(const EntityId& l_entity,
    const Direction& l_dir)
{
    EntityManager* entities = m_systemManager->GetEntityManager();
    if (!entities->HasComponent(l_entity,
        Component::SpriteSheet))
    {
        return;
    }
    C_SpriteSheet* sheet = entities->
        GetComponent<C_SpriteSheet>(l_entity, Component::SpriteSheet);
    sheet->GetSpriteSheet()->SetDirection(l_dir);
}
```

The `SetSheetDirection` method simply fetches the sprite sheet component and changes its direction.

The last feature we want to work on implementing is the render order of drawables in order to simulate depth. Drawing entities in a correct sequence requires them being sorted. This is where the `SortDrawables` method comes in:

```
void S_Renderer::SortDrawables() {
    EntityManager* e_mgr = m_systemManager->GetEntityManager();
    std::sort(m_entities.begin(), m_entities.end(),
        [e_mgr](unsigned int l_1, unsigned int l_2)
        {
            auto pos1 = e_mgr->
                GetComponent<C_Position>(l_1, Component::Position);
            auto pos2 = e_mgr->
                GetComponent<C_Position>(l_2, Component::Position);
            if (pos1->GetElevation() == pos2->GetElevation()) {
                return pos1->GetPosition().y < pos2->GetPosition().y;
            }
            return pos1->GetElevation() < pos2->GetElevation();
        });
}
```

Here, we simply invoke the `std::sort` function, with the last argument being the predicate lambda that we've already seen before. Elevation is given top priority when entity sprites are being sorted. Anything with a higher elevation will be drawn on top, while sprites on the same elevation are sorted based on their Y coordinate.

With that, the rendering system is now complete! Putting all of the pieces together is the final step in employing the entity component system pattern in our game.

Putting the ECS to work

Due to the part this paradigm plays in the overall structure of our application, we want the system manager and entity manager classes to be accessible to the majority of our code base. Having these objects be part of the shared context is the best way to do so:

```
struct SharedContext {
    SharedContext():
        ...
        m_systemManager(nullptr),
        m_entityManager(nullptr),
        ...{}
    ...
    SystemManager* m_systemManager;
```

```

    EntityManager* m_entityManager;
    ...
};

```

Adjusting the shared context means we have two extra classes to keep track of in `Game.h`:

```

class Game{
...
private:
    ...
    SystemManager m_systemManager;
    EntityManager m_entityManager;
    ...
};

```

These classes have to be properly initialized, which is done in `Game.cpp`:

```

Game::Game(): m_window("Chapter 8", sf::Vector2u(800,600)),
    m_entityManager(&m_systemManager, &m_textureManager),
    m_stateManager(&m_context)
{
    ...
    m_systemManager.SetEntityManager(&m_entityManager);
    m_context.m_systemManager = &m_systemManager;
    m_context.m_entityManager = &m_entityManager;
    ...
}

```

Notice that the entity manager is initialized in the initializer list. The system manager is then given a pointer to the entity manager, and both of these classes are added to the shared context.

Next up, some changes have to be made to the game state:

```

class State_Game : public BaseState{
public:
    ...
    void PlayerMove(EventDetails* l_details);
    ...
private:
    ...
    void UpdateCamera();
    int m_player;
};

```

The game state currently keeps track of the player's entity identifier in addition to providing a new method for updating the camera, as well as moving the player, which will be set up as a callback like so:

```
void State_Game::OnCreate() {
    ...
    evMgr->AddCallback(StateType::Game, "Player_MoveLeft",
        &State_Game::PlayerMove, this);
    evMgr->AddCallback(StateType::Game, "Player_MoveRight",
        &State_Game::PlayerMove, this);
    evMgr->AddCallback(StateType::Game, "Player_MoveUp",
        &State_Game::PlayerMove, this);
    evMgr->AddCallback(StateType::Game, "Player_MoveDown",
        &State_Game::PlayerMove, this);
    ...
    m_player = m_gameMap->GetPlayerId();
}
```

After the game map is loaded, the player entity identifier is obtained through the Map class, which stores this information during map loading.

The next task is getting the camera to follow our hero. This can be accomplished by first calling our `UpdateCamera` method in the `Update` method of our game state:

```
void State_Game::Update(const sf::Time& l_time) {
    SharedContext* context = m_stateMgr->GetContext();
    UpdateCamera();
    m_gameMap->Update(l_time.asSeconds());
    context->m_systemManager->Update(l_time.asSeconds());
}
```

The actual `UpdateCamera` method itself is implemented like so:

```
void State_Game::UpdateCamera() {
    if (m_player == -1) { return; }
    SharedContext* context = m_stateMgr->GetContext();
    C_Position* pos = m_stateMgr->GetContext()->m_entityManager->
        GetComponent<C_Position>(m_player, Component::Position);

    m_view.setCenter(pos->GetPosition());
    context->m_wind->GetRenderWindow()->setView(m_view);

    sf::FloatRect viewSpace = context->m_wind->GetViewSpace();
    if (viewSpace.left <= 0) {
        m_view.setCenter(viewSpace.width / 2, m_view.getCenter().y);
        context->m_wind->GetRenderWindow()->setView(m_view);
    }
}
```

```

    } else if (viewSpace.left + viewSpace.width >
               (m_gameMap->GetMapSize().x) * Sheet::Tile_Size)
    {
        m_view.setCenter(
            ((m_gameMap->GetMapSize().x) * Sheet::Tile_Size) -
            (viewSpace.width / 2), m_view.getCenter().y);
        context->m_wind->GetRenderWindow()->setView(m_view);
    }

    if (viewSpace.top <= 0){
        m_view.setCenter(m_view.getCenter().x, viewSpace.height / 2);
        context->m_wind->GetRenderWindow()->setView(m_view);
    } else if (viewSpace.top + viewSpace.height >
               (m_gameMap->GetMapSize().y) * Sheet::Tile_Size)
    {
        m_view.setCenter(m_view.getCenter().x,
            ((m_gameMap->GetMapSize().y) * Sheet::Tile_Size) -
            (viewSpace.height / 2));
        context->m_wind->GetRenderWindow()->setView(m_view);
    }
}

```

The player identifier first is verified of being a non-negative value, which would signify an error. The position component of the player entity is then obtained and used for updating the position of the current view. The rest of the code deals with adjusting the view to fit within the boundaries of the map, if it wanders outside of it. This is also where the system manager update method must be invoked.

Drawing our game world also needs revising:

```

void State_Game::Draw(){
    for(unsigned int i = 0; i < Sheet::Num_Layers; ++i){
        m_gameMap->Draw(i);
        m_stateMgr->GetContext()->m_systemManager->Draw(
            m_stateMgr->GetContext()->m_wind, i);
    }
}

```

First, a `for` loop iterates over each layer that might be used in the game. The `Num_Layers` value is part of the `Sheet` enumeration, which is defined in the `Map` class header. We'll be covering that shortly. The map `Draw` method now needs to know which layer to draw, as they're not all drawn at the same time anymore. After the appropriate layer is rendered, all of the entities that occupy the same elevation are also rendered on screen, giving the impression of depth in the game, like so:



Lastly, we need to define the callback method for moving a player:

```
void State_Game::PlayerMove(EventDetails* l_details){
    Message msg((MessageType)EntityMessage::Move);
    if (l_details->m_name == "Player_MoveLeft"){
        msg.m_int = (int)Direction::Left;
    } else if (l_details->m_name == "Player_MoveRight"){
        msg.m_int = (int)Direction::Right;
    } else if (l_details->m_name == "Player_MoveUp"){
        msg.m_int = (int)Direction::Up;
    } else if (l_details->m_name == "Player_MoveDown"){
        msg.m_int = (int)Direction::Down;
    }
    msg.m_receiver = m_player;
    m_stateMgr->GetContext()->m_systemManager->
        GetMessageHandler()->Dispatch(msg);
}
```

A message of type `Move` is created and set up in order to hold the direction in its `m_int` data member. The receiver of the message is also set to be the player, and the message is dispatched through the system manager's message handler. This message will be handled by one of the systems we'll be building in a later chapter.

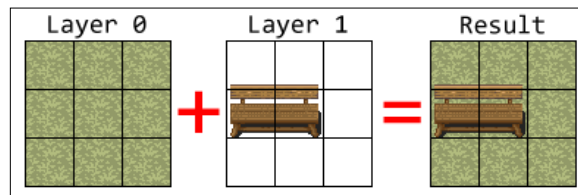
The last change from our previous project is the number of directions an entity can move in. Given the format of our new entity spritesheets, let's modify `Directions.h`:

```
enum class Direction{ Up = 0, Left, Down, Right };
```

Since the direction is used as a way to offset numbers of rows in sprite sheets for obtaining correct animations, the values set here are important. This small change concludes building and setting up the core of our component entity system! All that's left now is adjusting the `Map` class to satisfy and complement the new, shiny features of our game.

The new and improved map

For as good as the second project of this book looked, a lot of things about it were fairly primitive. Among its other shortcomings, the map design lacked complexity due to its inability to support tile layers. Having a more complex scene requires tiles being able to layer over each other, in a manner best represented by this illustration:



Adding layer support, as well as loading entity information after re-designing the way entities are handled requires some changes to be made to the map file format. Let's take a look at an example:

```
SIZE 32 32
DEFAULT_FRICTION 1.0 1.0
|ENTITY|Name|x|y|elevation|
ENTITY Player 256.0 256.0 1
...
|TILE|ID|x|y|layer|solid|
TILE 3 0 0 0 0
...
```

While some map properties remain in place, things like gravity or background images have been removed, as it no longer suits the genre of the game we're making. The main changes here are the entity and tile lines.

Loading an entity is as simple as providing the name of its entity file and a few pieces of data, relevant to maps, such as its position and elevation.

Tile loading is also slightly different now. In addition to its identifier and position, a tile now also requires having a layer, as well as a flag for solidity, which will be covered more in depth in the next chapter.

Amongst some of the bigger changes, a new value inside the *Sheet* enumeration is defined. It represents the maximum possible number of layers in any given map:

```
enum Sheet{
    Tile_Size = 32, Sheet_Width = 256,
    Sheet_Height = 256, Num_Layers = 4
};
```

Also, in order to allow individual solidity options, each tile now carries a solidity flag that can be turned on or off:

```
struct Tile{
    ...
    bool m_solid; // Is the tile a solid.
};
```

Working with an extra piece of information, the tile layer, requires certain modifications to be made to the `GetTile` and `ConvertCoords` methods:

```
class Map{
public:
    ...
    Tile* GetTile(unsigned int l_x, unsigned int l_y,
        unsigned int l_layer);
    ...
    void Draw(unsigned int l_layer);
private:
    unsigned int ConvertCoords(unsigned int l_x, unsigned int l_y,
        unsigned int l_layer) const;
    ...
    int m_playerId;
    ...
};
```

Note the `m_playerId` data member. It keeps track of what entity ID the player has been given after loading the map file.

Adjusting the Map class

It's time to start implementing all of these changes! First, let's take a look at the method used for obtaining map tiles:

```
Tile* Map::GetTile(unsigned int l_x, unsigned int l_y,
    unsigned int l_layer)
{
    if(l_x < 0 || l_y < 0 || l_x >= m_maxMapSize.x ||
        l_y >= m_maxMapSize.y || l_layer < 0 ||
        l_layer >= Sheet::Num_Layers)
    {
        return nullptr;
    }
    auto itr = m_tileMap.find(ConvertCoords(l_x,l_y,l_layer));
    if (itr == m_tileMap.end()){ return nullptr; }
    return itr->second;
}
```

The biggest difference here is the check for accessing tiles that are out of boundaries of the map. The method itself takes an additional argument that represents the tile layer, which it then passes into the `ConvertCoords` method. Working with tile layers would require having a third dimension added to a 2D array of tiles. Since we're storing all of this information in a one-dimensional array, some additional math has to be done in order to perform the conversion:

```
unsigned int Map::ConvertCoords(unsigned int l_x,
    unsigned int l_y, unsigned int l_layer) const
{
    return ((l_layer*m_maxMapSize.y+l_y) * m_maxMapSize.x + l_x);
}
```

If you previously visualized the map as a 2D grid, it's now becoming a 3D cube, with the layer value representing its depth.

The desired functionality of the updated `Draw` method has been outlined quite clearly in the game state `Draw` method. Let's implement that:

```
void Map::Draw(unsigned int l_layer){
    if (l_layer >= Sheet::Num_Layers){ return; }
    sf::RenderWindow* l_wind = m_context->m_wind->GetRenderWindow();
    sf::FloatRect viewSpace = m_context->m_wind->GetViewSpace();

    sf::Vector2i tileBegin(
        floor(viewSpace.left / Sheet::Tile_Size),
```

```
        floor(viewSpace.top / Sheet::Tile_Size));
    sf::Vector2i tileEnd(
        ceil((viewSpace.left + viewSpace.width) / Sheet::Tile_Size),
        ceil((viewSpace.top + viewSpace.height) / Sheet::Tile_Size));

    unsigned int count = 0;
    for(int x = tileBegin.x; x <= tileEnd.x; ++x){
        for(int y = tileBegin.y; y <= tileEnd.y; ++y){
            Tile* tile = GetTile(x,y,l_layer);
            if (!tile){ continue; }
            sf::Sprite& sprite = tile->m_properties->m_sprite;
            sprite.setPosition(x * Sheet::Tile_Size,
                y * Sheet::Tile_Size);
            l_wind->draw(sprite);
            ++count;
        }
    }
}
```

Before we get into any actual rendering, we must make sure that the provided layer argument does not exceed the defined maximum. Aside from that, the only real difference here is that we're passing the layer argument into the `GetTile` method now. That's quite a simple adjustment.

Lastly, the way tiles and entities are loaded has to be fixed. Let's take a look at snippets from the `LoadMap` method:

```
if(type == "TILE"){
    ...
    sf::Vector2i tileCoords;
    unsigned int tileLayer = 0;
    unsigned int tileSolidity = 0;
    keystream >> tileCoords.x >> tileCoords.y >>
        tileLayer >> tileSolidity;
    if (tileCoords.x > m_maxMapSize.x ||
        tileCoords.y > m_maxMapSize.y ||
        tileLayer >= Sheet::Num_Layers)
    {
        std::cout << "! Tile is out of range: " <<
            tileCoords.x << " " << tileCoords.y << std::endl;
        continue;
    }
    Tile* tile = new Tile();
```

```

// Bind properties of a tile from a set.
tile->m_properties = itr->second;
tile->m_solid = (bool)tileSolidity;
if(!m_tileMap.emplace(ConvertCoords(
    tileCoords.x,tileCoords.y,tileLayer),tile).second)
{
    ...
}
...
} else if ...

```

The majority of this code remains unchanged. Reading in the layer and solidity data has been added, as well as checking if the layer value is valid, together with the coordinate values. The entity side of things, however, is quite different:

```

} else if(type == "ENTITY"){
    // Set up entity here.
    std::string name;
    keystore >> name;
    if (name == "Player" && m_playerId != -1){ continue; }
    int entityId = m_context->m_entityManager->AddEntity(name);
    if (entityId < 0){ continue; }
    if(name == "Player"){ m_playerId = entityId; }
    C_Base* position = m_context->m_entityManager->
        GetComponent<C_Position>(entityId,Component::Position);
    if(position){ keystore >> *position; }
} else ...

```

The name of an entity is read in first. If it's a player entity and it hasn't yet been set up based on the `m_playerId` data member, or if it's just any other entity, an attempt is made to add it. Upon successfully doing so, its name is checked once more in order to make sure the player entity identifier is caught and stored. The position component is then obtained and its data is read in directly from the map file.

Once the `Map` class is finished, compiling and rendering our project and loading in a valid map should leave us with a few entities standing around peacefully:



Summary

With the invention of all the tools we need, we will next be working on adding the most common game elements to our final project and bringing it to life, not to mention actually flexing the backend functionality we built. Although this chapter has come to an end, this is by no means the last of us discovering and applying new programming patterns, should a need ever arise again to use one.

A good code-base is one that can handle new features and expansion of old ones with ease. The fruition of this chapter marks the point, at which the games we make are no longer bound to be simplistic due to design restrictions or inconvenient expansion. At this point, the question is no longer "How?", it's "Why not?" Seeing as you have made it this far, why not keep going? See you in the next chapter!